

Universidade Federal de Santa Catarina
Centro de Ciências, Tecnologias e Saúde
Campus Araranguá
Curso de Engenharia de Computação

Controle de Acesso Inteligente para Imóveis de Aluguel por Temporada Baseado em IoT com ESP32 e MQTT

Modelo de Proposta de Projeto

Projeto de Sistemas Ubíquos e Embarcados — 2026.1

Alunos:

1. Fernando Doqui Futila

Araranguá, maio de 2026

Sumário

1	Introdução	2
1.1	Contextualização e Relevância do Tema	2
1.2	Justificativa	2
1.3	Problemática	2
1.4	Objetivo Geral	3
2	Trabalhos Relacionados	3
3	Requisitos do Sistema	5
3.1	Requisitos Funcionais	5
3.2	Requisitos Não Funcionais	6
4	Proposta / Desenvolvimento	7
4.1	Descrição da Solução	7
4.2	Diagrama Esquemático da Arquitetura do Sistema	8
4.3	Diagrama Entidade-Relacionamento do Banco de Dados	8
4.4	Diagrama de Estados do Acesso Temporário	9
4.5	Diagrama de Sequência — Abertura da Porta pelo Hóspede	9
4.6	Ferramentas de Hardware e Software	11
4.6.1	Hardware	11
4.6.2	Software	12
4.7	Fatores de Risco	13

1 Introdução

1.1 Contextualização e Relevância do Tema

O fenômeno da economia compartilhada, consolidado globalmente por plataformas como Airbnb e Booking.com, transformou unidades residenciais convencionais em ativos de hotelaria dinâmica. Essa transição, no entanto, impõe desafios de infraestrutura significativos, uma vez que os sistemas de controle de acesso tradicionais não possuem a granularidade necessária para gerenciar hóspedes de curta duração.

A automação residencial, antes vista apenas como um item de luxo, tornou-se uma ferramenta de gestão essencial para garantir a viabilidade operacional e a segurança desses imóveis. Neste contexto, a Internet das Coisas (IoT) surge como o elo tecnológico capaz de resolver essa lacuna, permitindo a digitalização da barreira física através de dispositivos conectados à rede mundial de computadores.

Os sistemas ubíquos e embarcados são protagonistas nesse cenário: microcontroladores de baixo custo como o ESP32 combinam conectividade Wi-Fi, capacidade de processamento local e interface com sensores físicos em um único chip, viabilizando soluções que antes exigiam hardware proprietário de alto custo.

1.2 Justificativa

A justificativa deste trabalho fundamenta-se na carência de soluções que integrem, em um único ecossistema de baixo custo, o controle de acesso temporário e o monitoramento ambiental reativo. Embora existam protótipos acadêmicos e produtos comerciais de alto custo, poucos conseguem atender simultaneamente aos requisitos de:

1. Geração e expiração automática de credenciais vinculadas ao calendário de reservas;
2. Gestão eficiente de múltiplos usuários e propriedades;
3. Integração nativa com sensores de segurança (gás, fumaça e temperatura);
4. Baixo custo de replicação através de hardware *open-source*.

A maioria das fechaduras inteligentes comerciais é voltada para uso residencial fixo, apresentando limitações críticas para o modelo de aluguel: incapacidade de gerar credenciais temporárias automáticas, necessidade de interação presencial para configuração e ausência de integração com sensores ambientais que monitorem a integridade do imóvel durante a estadia.

1.3 Problemática

O problema central reside na falta de interoperabilidade e na dependência de conectividade constante. Sistemas baseados puramente em nuvem falham quando a rede local do imóvel

apresenta instabilidade, enquanto sistemas puramente locais (como teclados de senha fixa) não permitem a revogação dinâmica de acessos.

Além disso, a segurança patrimonial em imóveis alugados é frequentemente negligenciada: vazamentos de gás ou princípios de incêndio podem ocorrer sem que o proprietário seja notificado ou que o sistema de acesso tome medidas automáticas para facilitar a evacuação ou o socorro.

A gestão de chaves físicas ainda é uma realidade para muitos anfitriões, gerando:

- Custos logísticos de deslocamento para entrega e recolhimento de chaves;
- Riscos de duplicação não autorizada;
- Total ausência de auditoria sobre quem acessa a propriedade e em quais horários.

1.4 Objetivo Geral

Este projeto tem como objetivo desenvolver e validar uma solução de controle de acesso inteligente baseada em IoT, voltada especificamente para imóveis de aluguel por temporada, integrando o gerenciamento remoto de chaves digitais e o monitoramento preventivo de sensores ambientais, por meio de um dispositivo embarcado de baixo custo (ESP32) conectado a uma plataforma *backend* Django via protocolo MQTT sobre HiveMQ Cloud.

2 Trabalhos Relacionados

A pesquisa sobre fechaduras inteligentes avançou significativamente com a popularização de microcontroladores de baixo custo. Contudo, a aplicação voltada especificamente para o mercado de locação por temporada ainda apresenta lacunas operacionais. A seguir são apresentados os principais trabalhos relacionados identificados na literatura.

Trabalho 1 — Smart Lock System Using BLE and Multi-Factor Authentication

Proposta de arquitetura centrada em Bluetooth Low Energy (BLE) que exige autenticação multifator para liberar a porta. Embora ofereça alta segurança contra clonagem de sinal, a dependência da proximidade física do hóspede dificulta a gestão remota e a auditoria em tempo real, uma vez que os registros não são sincronizados instantaneamente com a nuvem. Não há integração com sensores ambientais.

Trabalho 2 — Temporary QR Code-Based Access Control for Short-Term Rentals

Propõe o uso de QR Codes temporários gerados pelo servidor para abertura de portas. A principal vantagem é a capacidade de operar *offline*, mas a solução carece de um mecanismo de revogação imediata: caso um hóspede precise ser impedido de entrar antes do fim do prazo, o código físico ou digital em sua posse continuaria válido até a expiração local. Não há monitoramento ambiental.

Trabalho 3 — Retrofit de Fechaduras Tradicionais com Microcontroladores e Identificação por MAC Address

Foca no *retrofit* de fechaduras tradicionais, minimizando custos de instalação. Entretanto, a segurança baseada apenas no endereço MAC do dispositivo móvel é insuficiente para o mercado de aluguel profissional, dado que esse identificador pode ser falsificado por softwares comuns, e não há suporte a múltiplos usuários com janelas temporais distintas.

Trabalho 4 — IoT-Based Home Automation and Security System Using MQTT and ESP8266

Sistema de automação residencial baseado em MQTT com ESP8266 que integra controle de iluminação, temperatura e alarmes. Demonstra a viabilidade do protocolo MQTT para ambientes com recursos limitados, mas não trata o ciclo de vida de credenciais temporárias nem o conceito de "servidor interceptador"— os comandos podem ser publicados diretamente no broker por qualquer cliente autenticado, sem validação de regras de negócio.

Trabalho 5 — Secure Access Control System for IoT Using Token-Based Authentication and Cloud Broker

Propõe um sistema de controle de acesso IoT com autenticação por token JWT e broker MQTT em nuvem. A solução é mais próxima da proposta deste projeto, porém não contempla o monitoramento de sensores ambientais (gás, fumaça, temperatura) integrado ao mesmo dispositivo, nem o fluxo de resgate de credencial pelo hóspede sem criação de conta formal.

Ao comparar as soluções existentes, percebe-se que nenhuma integra simultaneamente o controle de acesso temporal dinâmico com a segurança reativa baseada em sensores ambientais. É nesta lacuna que o presente trabalho se posiciona, oferecendo uma plataforma integrada que prioriza tanto a conveniência do hóspede quanto a integridade do patrimônio do anfitrião.

3 Requisitos do Sistema

3.1 Requisitos Funcionais

#	Módulo	Descrição
RF01	Autenticação	O sistema deve permitir que anfitriões se registrem com nome, e-mail e senha, e façam login com geração de token de autenticação.
RF02	Gestão de fechaduras	O anfitrião deve poder cadastrar, editar e remover fechaduras eletrônicas, associadas por endereço MAC do ESP32.
RF03	Abertura remota	O anfitrião deve poder acionar a abertura da fechadura remotamente pela interface web, mediante publicação de comando MQTT.
RF04	Acesso temporário	O sistema deve gerar tokens de resgate de 12 caracteres hexadecimais, com janela temporal de validade definida (início e fim de reserva).
RF05	Ciclo de vida do acesso	O status do acesso temporário deve seguir a máquina de estados: PENDENTE → AGENDADO → ATIVO → EXPIRADO / REVOGADO.
RF06	Resgate de token pelo hóspede	O hóspede deve poder resgatar o token via aplicativo mobile sem necessidade de criar conta no sistema.
RF07	Abertura pelo hóspede	O sistema deve validar a janela temporal da reserva antes de publicar o comando de abertura, rejeitando acessos fora do período.
RF08	Revogação de acesso	O anfitrião deve poder revogar um acesso temporário ativo a qualquer momento, impedindo novas aberturas imediatamente.
RF09	Telemetria de sensores	O firmware deve publicar leituras dos sensores (temperatura, umidade, concentração de gás) a cada intervalo configurável.
RF10	Monitoramento ambiental	O sistema deve registrar alertas automáticos quando o sensor de gás ultrapassar limiar de risco, atuando de forma autônoma no firmware (abertura preventiva do relé) sem depender de conectividade.

#	Módulo	Descrição
RF11	Heartbeat e status online	O firmware deve publicar mensagem de heartbeat "ONLINE" a cada 30 segundos; a desconexão deve atualizar o campo <code>esta_online</code> via Last Will Testament (LWT).
RF12	Histórico de eventos	O sistema deve registrar todos os eventos de acesso, alertas de sensor e eventos de sistema em log auditável com timestamp.
RF13	Interface web (dashboard)	Deve existir painel web responsivo para anfitriões com telas de fechaduras, reservas, logs e perfil.
RF14	Aplicativo mobile	Deve existir aplicativo mobile para hóspedes (iOS e Android) para resgate de token e abertura da porta.

3.2 Requisitos Não Funcionais

#	Categoria	Descrição
RNF01	Segurança	Toda comunicação entre backend e broker MQTT deve utilizar TLS (porta 8883) com autenticação por usuário e senha.
RNF02	Segurança	Nenhuma interface de usuário (web ou mobile) deve se comunicar diretamente com o broker MQTT ou com o hardware (padrão Servidor Interceptador).
RNF03	Segurança	A API REST deve exigir token de autenticação em todos os endpoints de anfitrião; endpoints de hóspede autenticam-se pelo token_resgate.
RNF04	Precisão de dados	Valores de sensores devem ser preservados com no mínimo 4 algarismos significativos em toda a cadeia: firmware → MQTT → banco de dados → interface.
RNF05	Desempenho	A latência entre o acionamento do botão de abertura na interface e a atuação do relé no ESP32 deve ser inferior a 3 segundos em condições normais de rede.
RNF06	Disponibilidade	O firmware deve operar de forma autônoma para eventos de emergência (detecção de gás) independentemente da disponibilidade da rede.
RNF07	Escalabilidade	A arquitetura backend deve suportar múltiplos anfitriões, múltiplas fechaduras e múltiplos hóspedes simultâneos.

#	Categoria	Descrição
RNF08	Portabilidade	O backend deve ser independente de motor de banco de dados, suportando SQLite3 (desenvolvimento) e PostgreSQL (produção) sem alterações no código-fonte.
RNF09	Usabilidade	O dashboard web deve ser responsivo e funcional em resoluções a partir de 768px. O aplicativo mobile deve funcionar nos sistemas iOS e Android via Expo.
RNF10	Rastreabilidade	Todo evento de acesso, alerta ou sistema deve ser persistido com timestamp em <code>HistoricoEvento</code> para fins de auditoria.
RNF11	Custo de hardware	O custo total dos componentes de hardware por unidade instalada (ESP32 + sensores + relé) deve ser inferior a R\$100,00.

4 Proposta / Desenvolvimento

4.1 Descrição da Solução

A proposta consiste no desenvolvimento de um ecossistema de controle de acesso inteligente para imóveis de aluguel por temporada, composto por quatro camadas integradas: firmware embarcado em ESP32, servidor backend Django, painel web para anfitriões e aplicativo mobile para hóspedes.

O sistema adota a arquitetura denominada **Servidor Interceptador**: nenhuma interface de usuário se comunica diretamente com o hardware. Todo comando de abertura obrigatoriamente passa pelo servidor Django, que valida regras de negócio (identidade do solicitante, janela temporal da reserva) antes de publicar o payload "OPEN" no tópico MQTT correspondente ao dispositivo. O ESP32, que mantém subscrição nesse tópico via HiveMQ Cloud (TLS 8883), recebe o comando e aciona o relé da fechadura.

O fluxo completo de um acesso temporário envolve:

1. O anfitrião cadastra a fechadura com o endereço MAC do ESP32 pelo dashboard;
2. O anfitrião cria uma reserva, definindo o e-mail do hóspede e o período;
3. O sistema gera automaticamente um token de resgate de 12 caracteres hexadecimais;
4. O hóspede insere o token no aplicativo, que valida junto ao servidor (status: PENDENTE → AGENDADO);
5. Dentro da janela reservada, o hóspede pressiona "Abrir Porta"; o servidor valida a janela temporal e publica "OPEN" no tópico MQTT;

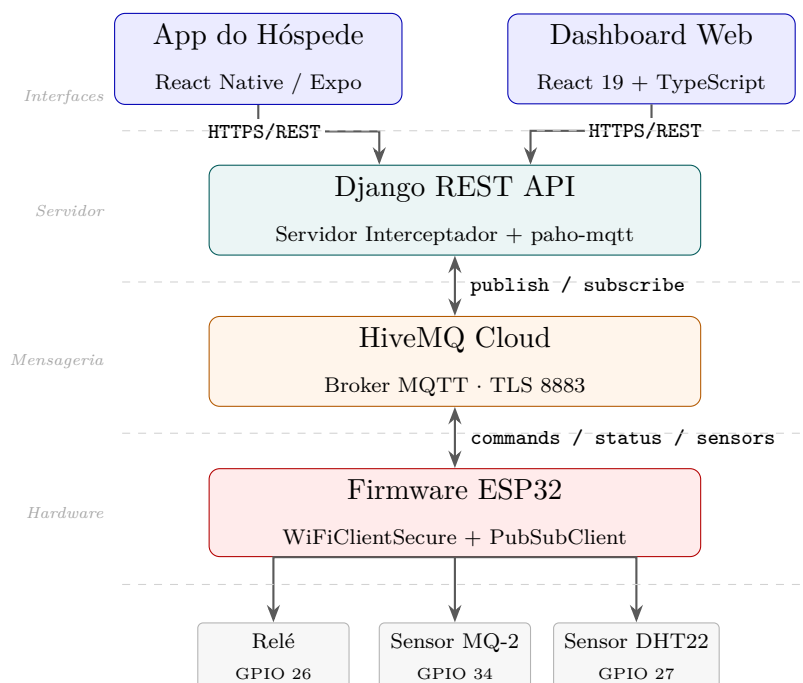
6. O ESP32 recebe o comando e aciona o relé; o evento é registrado no histórico.

Paralelamente, o firmware publica telemetria de sensores (temperatura, umidade, concentração de gás) a cada 30 segundos. Em caso de detecção de gás acima do limiar de risco, o firmware atua de forma autônoma — abrindo o relé para evacuação — sem depender da conectividade com o servidor.

4.2 Diagrama Esquemático da Arquitetura do Sistema

A Figura 1 apresenta o diagrama de arquitetura do sistema, ilustrando as quatro camadas (interfaces, servidor, mensageria e hardware) e os canais de comunicação entre elas.

Figura 1: Diagrama de arquitetura do sistema — padrão Servidor Interceptador.

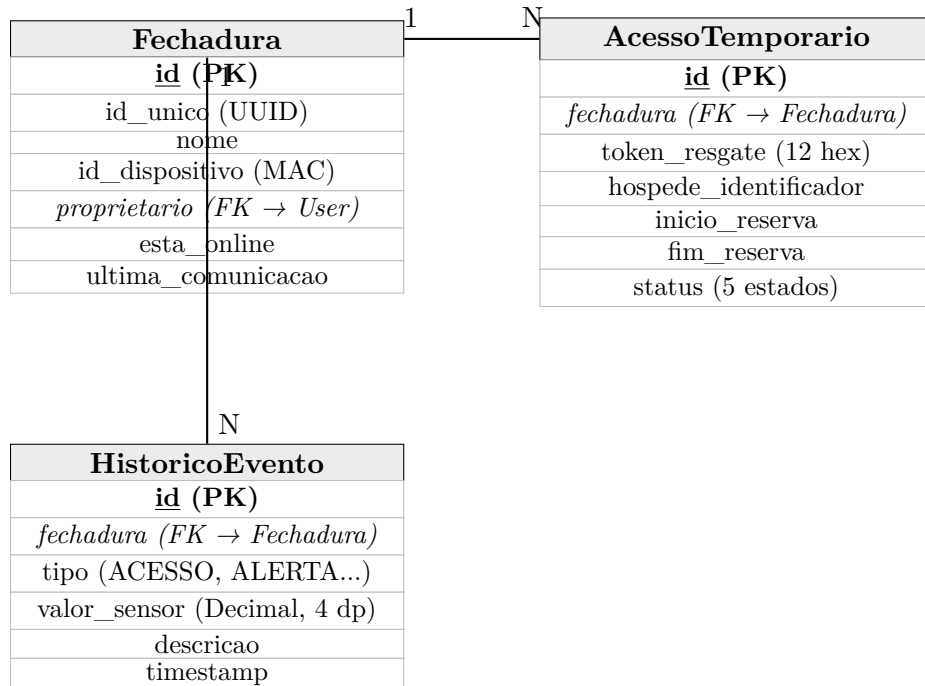


Fonte: Próprio autor.

4.3 Diagrama Entidade-Relacionamento do Banco de Dados

A Figura 2 apresenta o Diagrama Entidade-Relacionamento das três entidades centrais que sustentam a lógica do sistema: **Fechadura**, **AcessoTemporario** e **HistoricoEvento**.

Figura 2: Diagrama Entidade-Relacionamento do banco de dados.

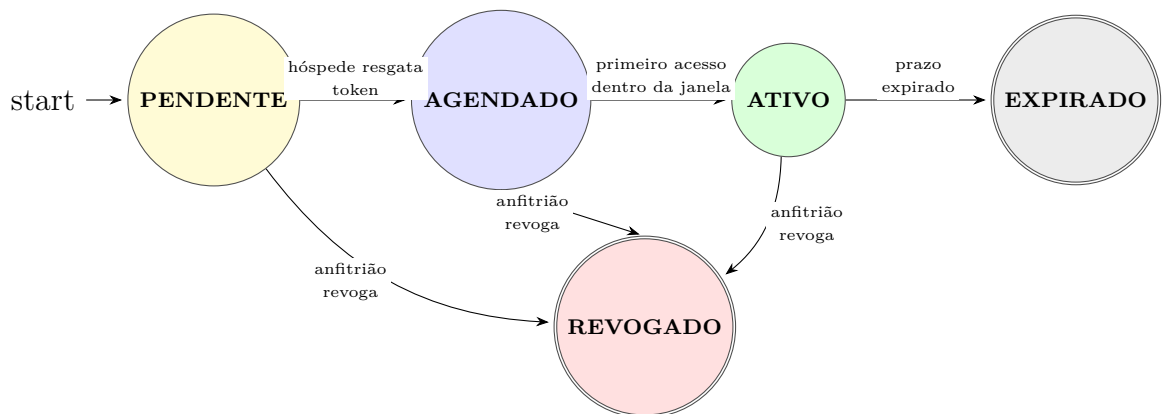


Fonte: Próprio autor.

4.4 Diagrama de Estados do Acesso Temporário

A Figura 3 apresenta a máquina de estados do **AcessoTemporario**, evidenciando as cinco transições possíveis ao longo do ciclo de vida de uma reserva.

Figura 3: Máquina de estados do AcessoTemporario.

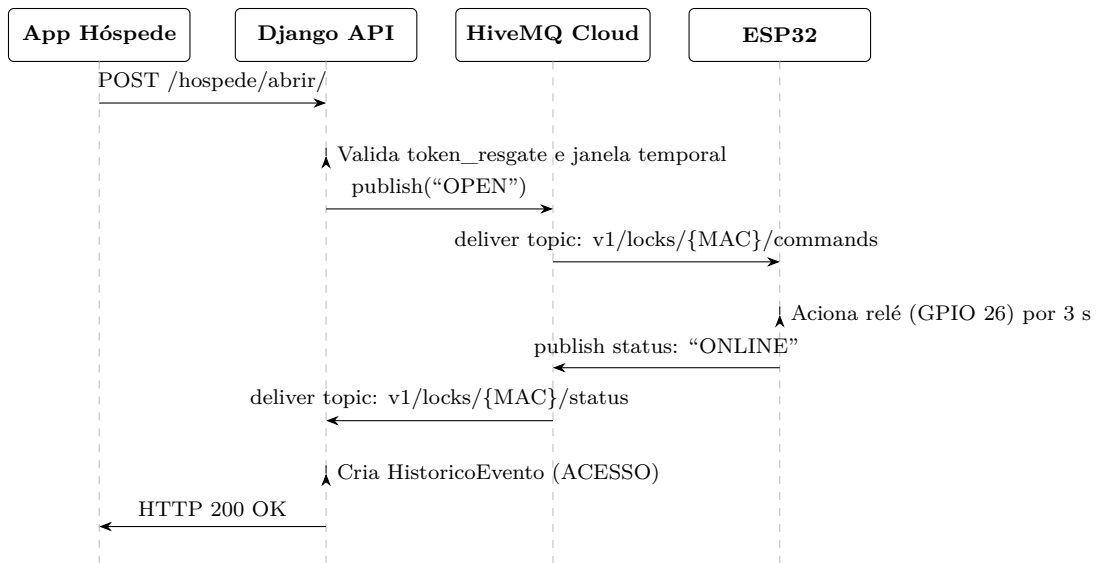


Fonte: Próprio autor.

4.5 Diagrama de Sequência — Abertura da Porta pelo Hóspede

A Figura 4 ilustra o fluxo de mensagens durante uma operação de abertura de porta solicitada pelo hóspede, desde o acionamento no aplicativo até a atuação do relé no ESP32.

Figura 4: Diagrama de sequência da abertura da porta pelo hóspede.



Fonte: Próprio autor.

4.6 Ferramentas de Hardware e Software

4.6.1 Hardware

Tabela 3: Componentes de hardware do dispositivo embarcado.

Componente	Modelo	Função
Microcontrolador	ESP32 (Espressif)	Processamento central, conectividade Wi-Fi, execução do firmware, controle dos periféricos via GPIO. Processador dual-core, opera em 3,3 V.
Sensor de gás/fumaça	MQ-2	Detecta concentrações de GLP, metano, CO e fumaça. Saída analógica lida via ADC no GPIO 34 do ESP32. Normalizado com 4 casas decimais.
Sensor de temp./umidade	DHT22	Mede temperatura (-40 a 80°C , $\pm 0,5^{\circ}\text{C}$) e umidade relativa (0–100%, $\pm 2\%$). Interface single-wire no GPIO 27.
Módulo relé	Relé 5V (1 canal)	Isola eletricamente o circuito de controle (3,3 V) do circuito de potência da fechadura solenoide. Acionado pelo GPIO 26 do ESP32.
Fechadura solenoide	Solenoide 12V	Atuador físico da barreira de acesso. Ativada pelo módulo relé mediante recebimento do comando <code>OPEN</code> .
LED indicador	LED 5mm (verde)	Sinalização visual de status da conexão e execução de comandos. GPIO 2.

Fonte: Próprio autor.

4.6.2 Software

Tabela 4: Tecnologias de software utilizadas no sistema.

Tecnologia	Versão / Tipo	Papel no sistema
C++ (Arduino)	Arduino Framework	Linguagem do firmware ESP32. Bibliotecas: PubSubClient (MQTT), DHT (sensor DHT22), ArduinoJson (serialização JSON), WiFiClientSecure (TLS).
Python Django	/ Django 5 + DRF	Backend REST API; modelos ORM; publicação MQTT via paho-mqtt; daemon mqtt_subscriber para telemetria em tempo real.
TypeScript React	/ React 19 + Vite	Dashboard web do anfitrião. Shadcn/UI, Tailwind CSS v4, Axios, React Router v6, React Hook Form + Zod.
React Native Expo	/ Expo SDK 54	Aplicativo mobile do hóspede (iOS e Android). expo-secure-store para persistência segura do token.
HiveMQ Cloud	Broker MQTT (SaaS)	Mensageria MQTT com TLS obrigatório na porta 8883 e autenticação por usuário/senha. Plano gratuito para prototipagem.
SQLite3 PostgreSQL	/ SQLite (dev)	Banco de dados relacional. SQLite3 para desenvolvimento local; PostgreSQL para produção (migração transparente via ORM Django).
Git / GitHub	Monorepo	Controle de versão e hospedagem.

Fonte: Próprio autor.

4.7 Fatores de Risco

Tabela 5: Fatores de risco identificados e respectivas estratégias de mitigação.

#	Risco	Categoria	Impacto	Mitigação
R01	Queda da conexão Wi-Fi no imóvel	Conectividade	Alto	Firmware trata emergências localmente sem servidor. Reconexão automática ao Wi-Fi em loop.
R02	Latência elevada no broker MQTT	Desempenho	Médio	HiveMQ Cloud com SLA de disponibilidade. MQTT assíncrono; meta <3s monitorada.
R03	Validação de certificado TLS ausente	Segurança	Médio	<code>setInsecure()</code> para prototipagem. Produção usará <code>setCACert()</code> com certificado root HiveMQ.
R04	Clonagem de token de resgate	Segurança	Alto	Token de 12 hex (72 bits de entropia). Após resgate, status muda para AGENDADO, impedindo reutilização.
R05	Imprecisão na leitura do sensor MQ-2	Precisão	Médio	Normalização com 4 casas decimais. Limiar de alerta calibrável por constante no firmware.
R06	Hardware físico indisponível para testes	Logístico	Alto	Firmware testável no simulador Wokwi. Software testável via API e dashboard sem hardware.
R07	SQLite3 inadequado para produção	Escalabilidade	Baixo	ORM Django suporta migração para PostgreSQL sem alterações no código.
R08	Status online sem WebSocket (polling)	Usabilidade	Baixo	LWT atualiza <code>esta_online</code> via <code>mqtt_subscriber</code> . WebSocket previsto como trabalho futuro.

Fonte: Próprio autor.